230724E_a2

October 19, 2026

0.0.1 EN3160 Assignment 2 on Fitting and Alignment

Index Number: 230724E

Name: Wijesekara WAGS

GitHub Link: SathsaraGeeth/*vision_a02*

0.1 1. Blob Detection

[1]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import maximum_filter

im = cv.imread("the_berry_farms_sunflower_field.jpeg", cv.
 ↪IMREAD_REDUCED_COLOR_4)

plt.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()

# ker_sz shall be an odd positive integer
def NormalizedLapOfGauss2D(sigma, ker_sz):
    x = np.arange(-(ker_sz // 2), ker_sz // 2 + 1)
    y = np.arange(-(ker_sz // 2), ker_sz // 2 + 1)
    X, Y = np.meshgrid(x, y)
    r2 = X**2 + Y**2
    kernel = (r2 - 2 * sigma**2) / (2 * np.pi * sigma**6)
    kernel *= np.exp(-r2 / (2 * sigma**2))
    kernel *= sigma**2
    kernel -= kernel.mean()

    return kernel
```

```
def BlobDetector(image, num_blobs=25, threshold_rel=0.03, sigma_min=1.5,␣
 ↪sigma_max=25, num_scales=30):

    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY).astype(np.float64)

    sigmas = np.linspace(sigma_min, sigma_max, num_scales)
    responses = []

    # Convolve image with scale-normalized Laplacian at several scales
    for sigma in sigmas:
        ker_sz = 2 * int(np.ceil(3 * sigma)) + 1
        kernel = NormalizedLapOfGauss2D(sigma, ker_sz)
        response = cv.filter2D(gray, cv.CV_64F, kernel, borderType=cv.
 ↪BORDER_REFLECT101)
        responses.append(response ** 2)

    # Find maxima of squared Laplacian response in scale-space
    scale_space = np.stack(responses, axis=0)
    local_max = maximum_filter(scale_space, size=(3, 3, 3))
    threshold = threshold_rel * scale_space.max()
    maxima = (scale_space == local_max) & (scale_space > threshold)

    maxima[0] = False
    maxima[-1] = False

    scale_idx, ys, xs = np.where(maxima)

    height, width = gray.shape
    candidates = []
    for s, y, x in zip(scale_idx, ys, xs):
        radius = np.sqrt(2) * sigmas[s]

        if x - radius < 0 or x + radius >= width:
            continue
        if y - radius < 0 or y + radius >= height:
            continue

        score = float(scale_space[s, y, x])
        candidates.append((int(x), int(y), float(radius), score))

    candidates.sort(key=lambda blob: blob[3], reverse=True)
    detected = []
    for x, y, radius, score in candidates:
        overlaps = any((x - px)**2 + (y - py)**2 < max(radius, pr)**2 for px,␣
 ↪py, pr in detected)
        if not overlaps:
            detected.append((x, y, radius))
```

```
    return detected[:num_blobs]

# Limit to 10 largest blobs out of strongest 25 detections
blobs = sorted(BlobDetector(im, num_blobs=25), key=lambda blob: blob[2],␣
 ↪reverse=True)[:10]

fig, ax = plt.subplots(figsize=(5, 5))
ax.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))

for index, (x, y, radius) in enumerate(blobs, start=1):
    ax.add_patch(plt.Circle((x, y), radius, color='red', fill=False))
    print(f"Blob {index}: center=({x}, {y}), radius={radius:.2f} pixels")


ax.axis("off")
plt.show()
```




```
Blob 1: center=(282, 338), radius=10.14 pixels
Blob 2: center=(179, 327), radius=9.00 pixels
Blob 3: center=(106, 255), radius=7.85 pixels
Blob 4: center=(178, 261), radius=7.85 pixels
```

```
Blob 5: center=(189, 240), radius=6.71 pixels
Blob 6: center=(161, 290), radius=6.71 pixels
Blob 7: center=(63, 243), radius=6.71 pixels
Blob 8: center=(274, 258), radius=6.71 pixels
Blob 9: center=(303, 239), radius=5.56 pixels
Blob 10: center=(241, 279), radius=5.56 pixels
```



The image was first converted to grayscale. It was then convolved with the scale-normalized Laplacian of Gaussian at 30 scales in the range $1.5 \leq \sigma \leq 25$. The blobs were found by looking for maxima of the squared Laplacian response in space and scale. Responses below 3% of the maximum response were ignored, together with detections touching the image boundary. Overlapping detections were removed, and the 10 largest circles from the 25 strongest detections were drawn. The radius of each blob was found using $r = \sqrt{2}\sigma$.

The detected circle parameters (x, y, r) are printed by the code.

0.2 2. Fit a Line and a Circle using RANSAC

[2]:
```python
# Generate noisy points for a line and a circle
import numpy as np
from scipy.optimize import minimize
from scipy import linalg
import matplotlib.pyplot as plt
```

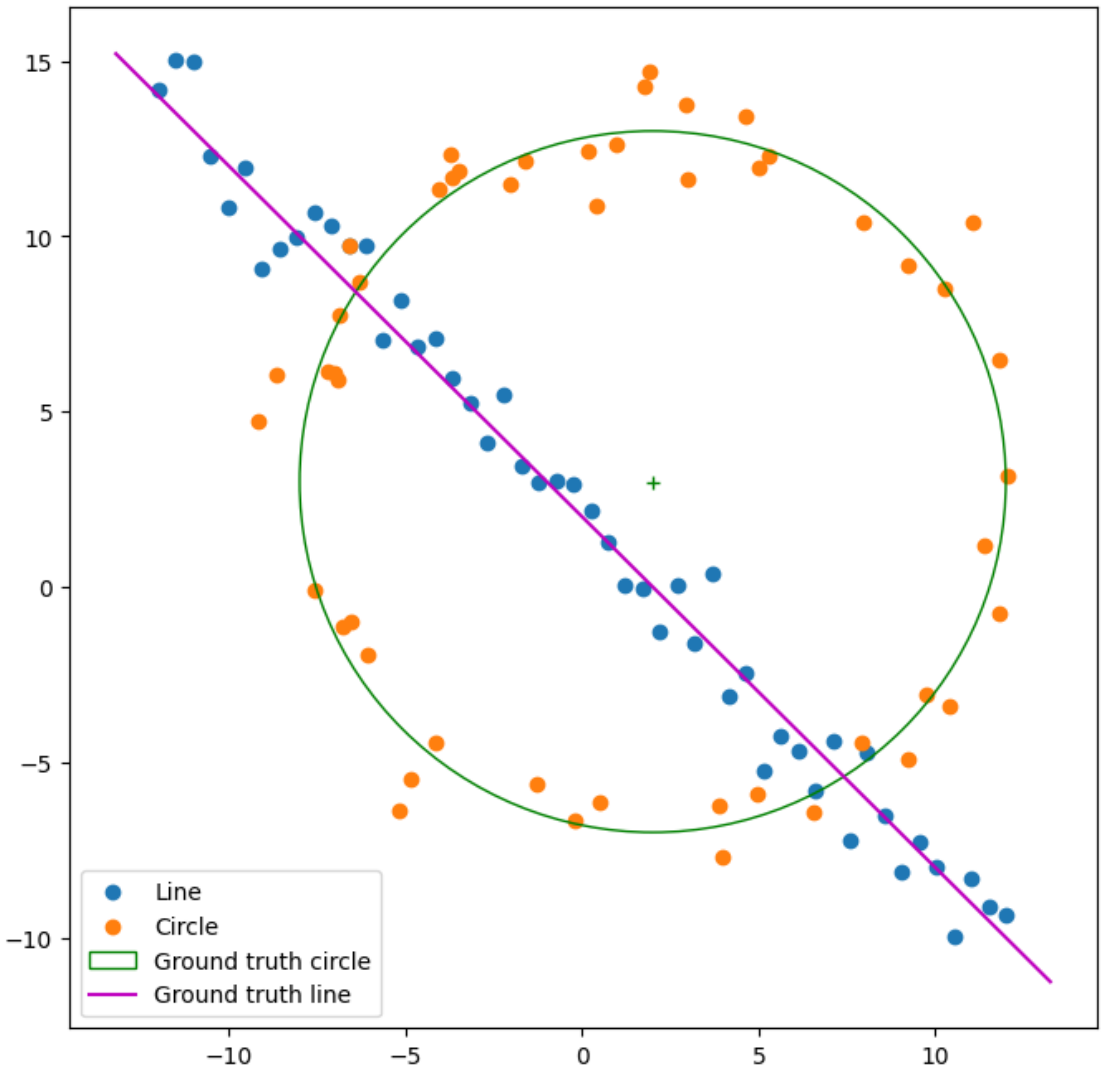
```
N = 100
half_n = N // 2

r = 10
x0_gt, y0_gt = 2, 3
s = r / 16
t = np.random.uniform(0, 2 * np.pi, half_n)
n = s * np.random.randn(half_n)
x = x0_gt + (r + n) * np.cos(t)
y = y0_gt + (r + n) * np.sin(t)
X_circ = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

s = 1.0
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m * x + b + s * np.random.randn(half_n)
X_line = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))
X = np.vstack((X_circ, X_line))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(X_line[:, 0], X_line[:, 1], label="Line")
ax.scatter(X_circ[:, 0], X_circ[:, 1], label="Circle")
circle_gt = plt.Circle((x0_gt, y0_gt), r, color="g", fill=False, label="Ground␣
 ↪truth circle")
ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, "+", color="g")

x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b
plt.plot(x_, y_, color="m", label="Ground truth line")
plt.legend()
plt.show()
```



0.2.1 (a) Line fitting using RANSAC

[3]:
```
# RANSAC settings
iterations = 1000
distance_threshold = 1.0
minimum_consensus = 35
rng = np.random.default_rng(0)
best_model = None
best_line_sample = None
best_inliers = np.array([], dtype=int)

for _ in range(iterations):
    i, j = rng.choice(len(X), 2, replace=False)
```

```
    p1, p2 = X[i], X[j]

    # Unit normal to the line through the two points
    a = p2[1] - p1[1]
    b = p1[0] - p2[0]
    length = np.sqrt(a**2 + b**2)
    if length == 0:
        continue
    a, b = a / length, b / length
    d = a * p1[0] + b * p1[1]
    if d < 0:
        a, b, d = -a, -b, -d

    # Normal distance from each point to the line
    errors = np.abs(a * X[:, 0] + b * X[:, 1] - d)
    inliers = np.where(errors < distance_threshold)[0]

    if len(inliers) >= minimum_consensus and len(inliers) > len(best_inliers):
        best_model = (a, b, d)
        best_line_sample = np.array([p1, p2])
        best_inliers = inliers

a, b, d = best_model
print(f"a = {a:.4f}, b = {b:.4f}, d = {d:.4f}")
print(f"||[a, b]|| = {np.sqrt(a**2 + b**2):.4f}")
print(f"Consensus = {len(best_inliers)} points")
```

```
a = 0.7259, b = 0.6878, d = 1.2982
||[a, b]|| = 1.0000
Consensus = 48 points
```

In each trial, two random points were used to calculate $ax + by = d$. The values of a and b were divided by $\sqrt{a^2 + b^2}$, so the error $|ax + by - d|$ is the normal distance to the line. The distance threshold was set to 1.0 and the minimum consensus was set to 35 points. Since the data contains 50 line points, this leaves some room for the added noise.

0.2.2 (b) Circle fitting using RANSAC

[4]:
```
# Remove the line consensus
remaining = np.delete(X, best_inliers, axis=0)

iterations = 1000
radial_threshold = 1.0
minimum_consensus = 35
rng = np.random.default_rng(1)
best_circle = None
best_circle_sample = None
best_circle_inliers = np.array([], dtype=int)
```

```
for _ in range(iterations):
    i, j, k = rng.choice(len(remaining), 3, replace=False)
    p1, p2, p3 = remaining[[i, j, k]]

    A = 2 * np.array([p2 - p1, p3 - p1])
    c = np.array([p2 @ p2 - p1 @ p1, p3 @ p3 - p1 @ p1])

    if abs(np.linalg.det(A)) < 1e-8:
        continue

    center = np.linalg.solve(A, c)
    radius = np.linalg.norm(p1 - center)

    # Radial error from each point to the circle
    errors = np.abs(np.linalg.norm(remaining - center, axis=1) - radius)
    inliers = np.where(errors < radial_threshold)[0]

    if len(inliers) >= minimum_consensus and len(inliers) >␣
 ↪len(best_circle_inliers):
        best_circle = (center[0], center[1], radius)
        best_circle_sample = np.array([p1, p2, p3])
        best_circle_inliers = inliers

x_center, y_center, radius = best_circle
print(f"x = {x_center:.4f}, y = {y_center:.4f}, r = {radius:.4f}")
print(f"Consensus = {len(best_circle_inliers)} points")
```

```
x = 1.8628, y = 3.2293, r = 10.1135
Consensus = 40 points
```

The inliers of the best line were removed first. Each circle trial used three random points from the remaining set. The radial error was $|\|p - c\| - r|$. The threshold was set to 1.0 and at least 35 points were required. The circle noise has a standard deviation of $10/16 = 0.625$, and the original set contains 50 circle points.

0.2.3 (c) RANSAC results

[5]:
```
# Best-fit line from the line inliers
line_points = X[best_inliers]
line_center = line_points.mean(axis=0)
_, _, V = linalg.svd(line_points - line_center)
a_fit, b_fit = V[-1]
a_fit, b_fit = np.array([a_fit, b_fit]) / np.sqrt(a_fit**2 + b_fit**2)
d_fit = a_fit * line_center[0] + b_fit * line_center[1]

# Best-fit circle from the circle inliers
circle_points = remaining[best_circle_inliers]
```

```
def circle_error(parameters):
    x_center, y_center, radius = parameters
    distances = np.linalg.norm(circle_points - [x_center, y_center], axis=1)
    return np.sum((distances - radius)**2)

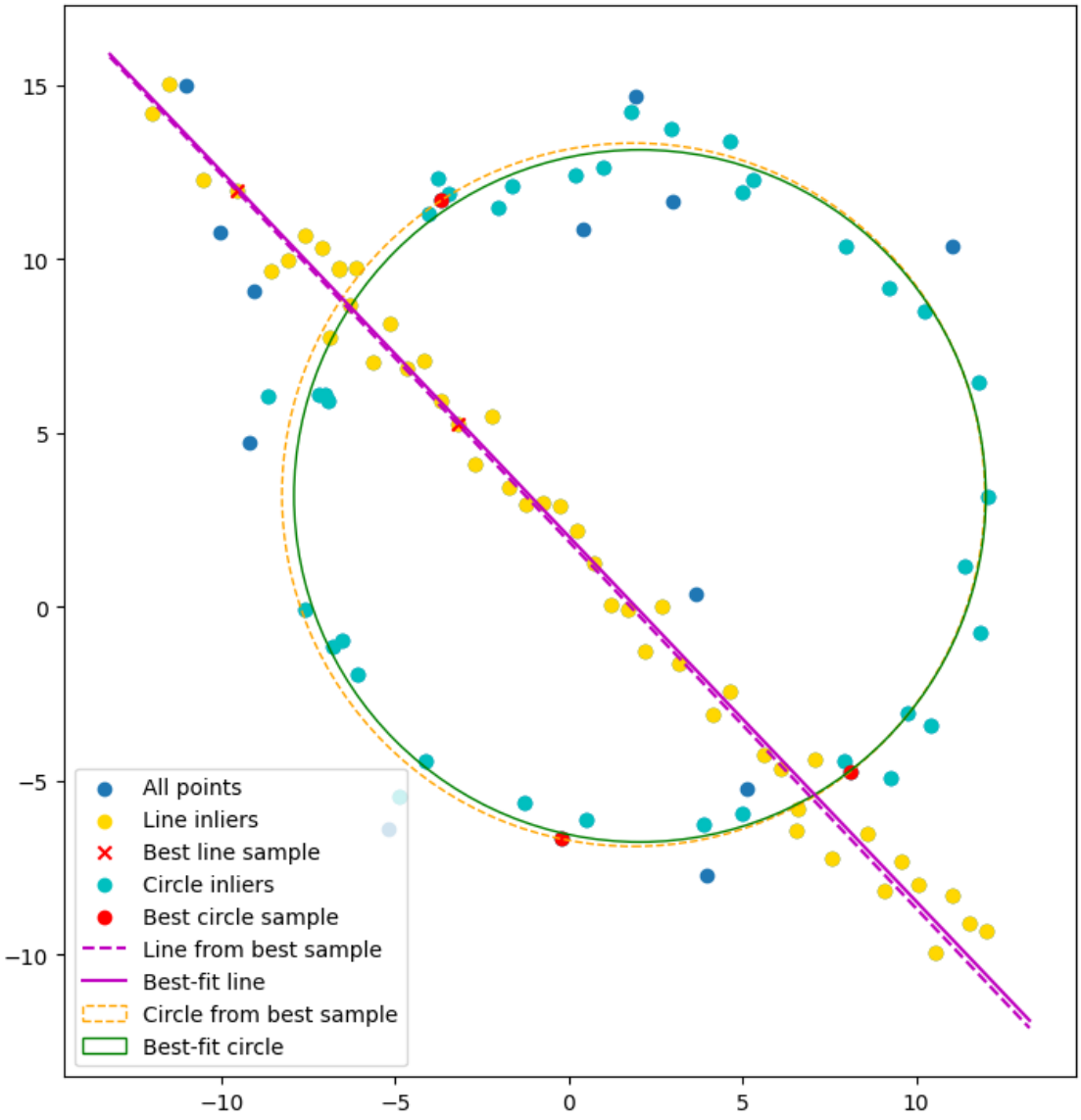
result = minimize(circle_error, best_circle)
x_fit, y_fit, r_fit = result.x

fig, ax = plt.subplots(figsize=(9, 9))
ax.scatter(X[:, 0], X[:, 1], color="tab:blue", label="All points")
ax.scatter(line_points[:, 0], line_points[:, 1], color="gold", label="Line␣
 ↪inliers")
ax.scatter(best_line_sample[:, 0], best_line_sample[:, 1], color="red",␣
 ↪marker="x", label="Best line sample")
ax.scatter(circle_points[:, 0], circle_points[:, 1], color="c", label="Circle␣
 ↪inliers")
ax.scatter(best_circle_sample[:, 0], best_circle_sample[:, 1], color="red",␣
 ↪label="Best circle sample")

x_values = np.array(ax.get_xlim())
ax.plot(x_values, (d - a * x_values) / b, color="m", linestyle="--",␣
 ↪label="Line from best sample")
ax.plot(x_values, (d_fit - a_fit * x_values) / b_fit, color="m",␣
 ↪label="Best-fit line")

sample_circle = plt.Circle(best_circle[:2], best_circle[2], color="orange",␣
 ↪fill=False, linestyle="--", label="Circle from best sample")
fitted_circle = plt.Circle((x_fit, y_fit), r_fit, color="green", fill=False,␣
 ↪label="Best-fit circle")
ax.add_patch(sample_circle)
ax.add_patch(fitted_circle)

ax.set_aspect("equal")
ax.legend()
plt.show()
```



0.2.4 (d) Fitting the circle first

If the circle is fitted first, the line points can give a circle with a very large radius. Part of a large circle looks almost straight, so those points may pass the radial-error test. They would then be removed as circle inliers, leaving too few points to fit the line properly.

0.3 3. Homography

[6]:
```
# load images
def add_flag(destination_path, source_path, destination_points):
    destination = cv.cvtColor(cv.imread(destination_path), cv.COLOR_BGR2RGB)
    source_file = cv.imread(source_path, cv.IMREAD_UNCHANGED)
```

```
    if source_file.shape[2] == 4:
        source = cv.cvtColor(source_file, cv.COLOR_BGRA2RGB)
        source_mask = source_file[:, :, 3]
    else:
        source = cv.cvtColor(source_file, cv.COLOR_BGR2RGB)
        source_mask = np.full(source.shape[:2], 255, dtype=np.uint8)

    # select 4 points from the pair
    destination_points = np.float32(destination_points)

    height, width = source.shape[:2]
    source_points = np.float32([
        [0, 0],
        [width - 1, 0],
        [width - 1, height - 1],
        [0, height - 1]
    ])

    # compute the homography
    H, _ = cv.findHomography(source_points, destination_points)

    # transform the source image to destination images coordinates
    destination_height, destination_width = destination.shape[:2]
    warped_source = cv.warpPerspective(source, H, (destination_width,␣
↪destination_height))

    # create a mask that identify the destination image covered by the source␣
↪image
    mask = cv.warpPerspective(source_mask, H, (destination_width,␣
↪destination_height))

    # blend them
    alpha = mask.astype(np.float32)[..., None] / 255.0
    result = destination * (1 - alpha) + warped_source * alpha
    result = result.astype(np.uint8)

    plt.figure(figsize=(10, 8))
    plt.imshow(result)
    plt.axis("off")
    plt.show()

    return result

pair_1 = add_flag(
    "homography_images/empty_playground.jpg",
    "homography_images/hopscotch_cropped.png",
    [[270, 350], [340, 350], [430, 470], [150, 470]]
```

```
)

pair_2 = add_flag(
    "homography_images/empty_billboard.jpg",
    "homography_images/starry_night.jpg",
    [[65, 60], [815, 48], [850, 425], [30, 435]]
)
```





For each pair, four points were selected on the flat surface of the destination image. These points were matched with the four corners of the source image to calculate the homography. The source image and its mask were then warped to the destination coordinates and blended with the destination image.

For the first pair, a hopscotch pattern was placed on the ground of an empty playground. This was chosen because the pattern belongs naturally in a playground, while the sloping ground makes the perspective change easy to see. For the second pair, The Starry Night art was placed on an empty billboard.

0.4 4. PCB Image Alignment

[7]:
```
# Load the template
template = cv.imread("AECC1.jpg")
template_gray = cv.cvtColor(template, cv.COLOR_BGR2GRAY)

# Find SIFT features in the template
sift = cv.SIFT_create()
kp1, des1 = sift.detectAndCompute(template_gray, None)
bf = cv.BFMatcher()

def align_pcb(filename):
    image = cv.imread(filename)
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
```

```
    # Find and match SIFT features
    kp2, des2 = sift.detectAndCompute(gray, None)
    matches = bf.knnMatch(des2, des1, k=2)
    good = [m for m, n in matches if m.distance < 0.75 * n.distance]

    # Find the homography using RANSAC
    src = np.float32([kp2[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp1[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv.findHomography(src, dst, cv.RANSAC, 5.0)

    inliers = [m for m, keep in zip(good, mask.ravel()) if keep]
    match_image = cv.drawMatches(image, kp2, template, kp1, inliers[:60], None,␣
 ↪flags=2)

    # Align the image and find the difference
    h, w = template.shape[:2]
    aligned = cv.warpPerspective(image, H, (w, h))
    valid = cv.warpPerspective(np.ones(gray.shape, np.uint8), H, (w, h))
    difference = cv.cvtColor(cv.absdiff(template, aligned), cv.COLOR_BGR2GRAY)
    difference[valid == 0] = 0

    return match_image, aligned, difference

images = ["AECC2.jpg", "AECC3.jpg", "AECC4.jpg"]

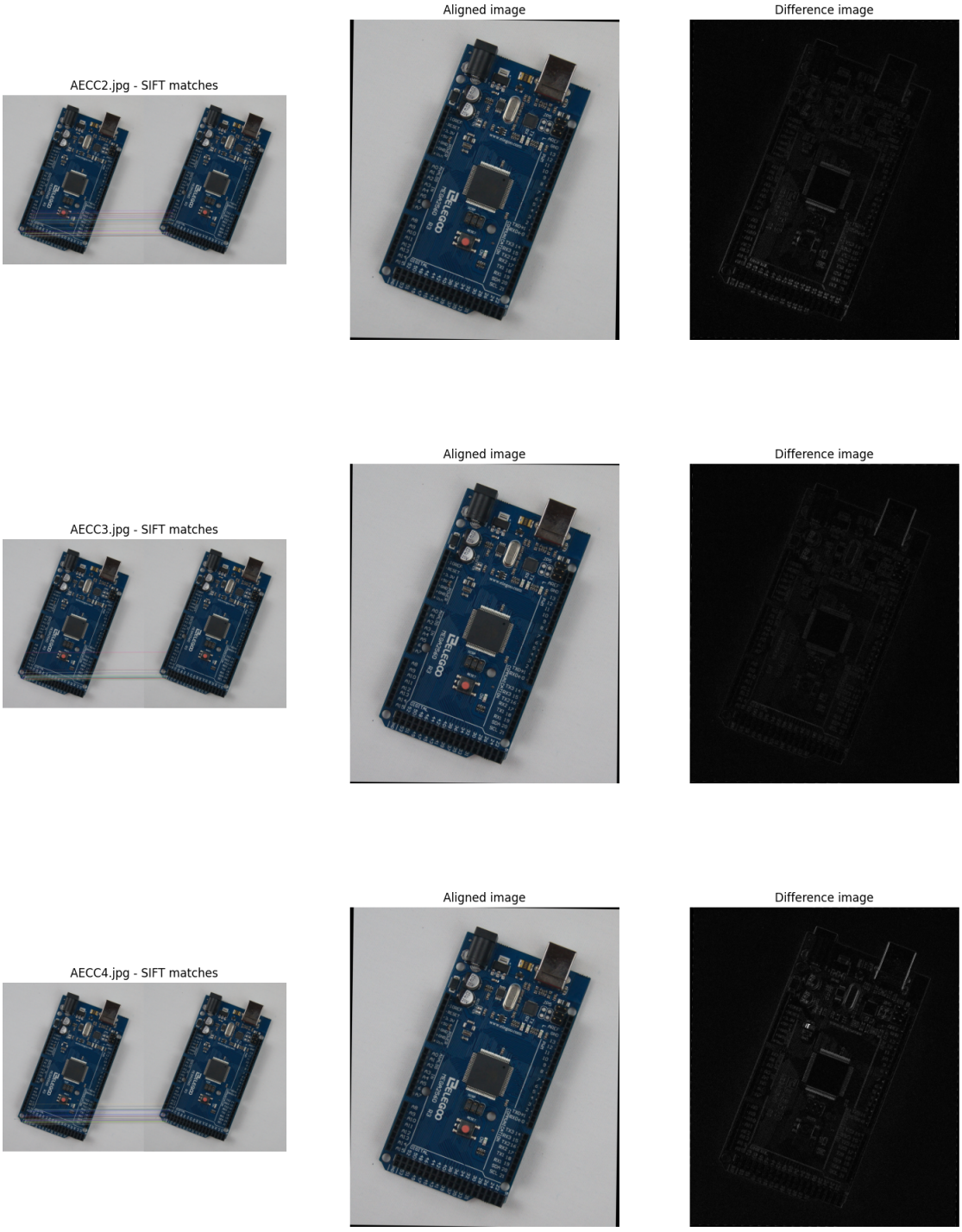
for filename in images:
    matches, aligned, difference = align_pcb(filename)

    plt.figure(figsize=(18, 6))
    plt.subplot(1, 3, 1)
    plt.imshow(cv.cvtColor(matches, cv.COLOR_BGR2RGB))
    plt.title(filename + " - SIFT matches")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(cv.cvtColor(aligned, cv.COLOR_BGR2RGB))
    plt.title("Aligned image")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(difference, cmap="gray", vmin=0, vmax=80)
    plt.title("Difference image")
    plt.axis("off")

    plt.show()
```



`AECC1.jpg` was used as the template. SIFT keypoints and descriptors were found in the template and in each of the other AECC images. The descriptors were matched using a brute-force matcher. The nearest match was kept when its distance was less than 0.75 times the distance of the second-nearest match. A homography was then fitted from these matches using RANSAC.

Each PCB image was warped to the template coordinates using the homography. The difference image was calculated using the absolute pixel difference between the aligned image and the template. The same steps were used for `AECC2.jpg`, `AECC3.jpg`, and `AECC4.jpg`. Bright areas in the difference images show places that do not agree with the template.

0.4.1 (e) Reason for high differences

The USB port, capacitors, and reset button are raised above the flat PCB. When the viewing angle changes, these parts move differently from the board surface because of parallax. A single homography can align the flat board, but it cannot fully align components at different heights. Changes in lighting and shadows also make these areas brighter in the difference image.